Session 5 – AADK Task 5

Understanding State in Compose

Student Name: Sachin
Course: Android App Development using Gen AI
Module: Jetpack Compose – State & Data Handling **Date:** _____

Task 1 – Deep Dive: Kotlin Functions, Lambdas & Higher-Order Functions

1. Function Types

1 Regular Named Function

```
fun greet(name: String): String {
return "Hello $name"
}
```

Use in Compose:
Used to separate UI logic like formatting display text.

Advantage:
Readable and easy to debug.

---

2 Lambda Expression

```
val greetLambda: (String) -> String = { name ->
   "Hello $name"
}
```

Use in Compose:
Commonly used for event handlers like `onClick = { }`.

Advantage:
Reduces boilerplate and keeps UI concise.

---

3 Higher-Order Function

```
fun applyOperation(a: Int, b: Int, operation: (Int, Int) -> Int): Int {
return operation(a, b)
}
```

Use in Compose:
Used when passing UI event callbacks or state update logic.

Advantage:
Makes functions flexible and reusable.

---

2. Comparison Table

Function Style	Syntax Example	When To Use	Trade-offs
Regular Function	fun greet()	Clear logic blocks	Slightly verbose
Lambda	{ name -> ... }	UI events, small logic	Can reduce readability if complex
Higher-Order	fun apply(op)	Reusable behavior	May confuse beginners

---

3. Mini Code Snippets

Extension Function Example

```
fun String.addExclamation(): String {
return this + "!" }
```

Usage:

```
println("Hello".addExclamation())
```

Name : Sachin
USN:1JS22CS133

Higher-Order Function in Compose Context

```
fun performAction(action: () -> Unit) {
action() }
```

Usage:

```
performAction {
   println("Button Clicked")
}
```

4. Reflection

Lambdas and higher-order functions reduce boilerplate in Compose by allowing concise event handling like `onClick = {}`. They help keep UI logic close to UI components. However, deeply nested lambdas may reduce readability and make debugging harder.

Task 2 – Designing a Kotlin Data Model

Feature: Task Tracker

Feature Description

This feature allows users to manage daily tasks. Each task has a title, description, and completion status. Users can mark tasks as completed.

Data Model

```
data class Task(
   val title: String,     val
description: String,     val
isCompleted: Boolean
)
```

Interface for Behavior

```
interface TaskManager {
   fun toggleTask(task: Task): Task
}
```

Why Use Data Class?

- Automatically generates equals(), copy(), toString()
- Supports immutability
- Cleaner code structure

Why Use Interface?

- Separates behavior from data
- Makes logic reusable
- Encourages clean architecture

Mapping to Compose UI

In Compose, tasks would be displayed using:

```
LazyColumn {
   items(taskList) { task ->
      Text(task.title)
   }
}
```

When a task is toggled, state updates and recomposition refreshes the UI.

Reflection

Designing data models before UI helps structure logic properly. Separating data and behavior improves maintainability. This makes Compose apps scalable and easier to manage.

Task 3 – State, Immutability & Side Effects

Definitions

Immutable Object:
An object whose state cannot change after creation.

Mutable State:
Data that can change during runtime.

Recomposition:

Process where Compose redraws UI when state changes.

Side Effect:
An operation that changes something outside its scope.

Mutable Example (Risky)

```
val tasks = mutableListOf<Task>()
tasks.add(Task("Study", "Compose basics", false))
```

Problem:
If state is not properly observed, UI may not update.

Immutable Example (Safer)

```
val tasks = listOf<Task>() val updatedTasks = tasks +
Task("Study", "Compose basics", false)
```

Using immutable lists ensures predictable UI updates.

State Management Plan

- Use immutable data classes
- Use `MutableState<List<Task>>`
- Avoid global mutable variables
- Use copy() to update objects
- Trigger recomposition by updating state

Reflection

Immutability ensures predictable behavior in UI apps. When state changes in Compose, recomposition updates only necessary components. Mutable shared state can cause inconsistent UI behavior. Using immutable patterns reduces bugs and improves reliability.

Name : Sachin
USN:1JS22CS133

Conclusion

This session helped me understand how Kotlin's functional features and immutability principles directly support Jetpack Compose's state-driven UI model. By designing structured data models and managing state properly, Compose applications become cleaner, more predictable, and easier to maintain.

Name : Sachin
USN:1JS22CS133